Martin Bergstø

Ontology-Driven Decision Support for Machine Learning Method Selection in Product Development and Production

Combining Semantic Literature Retrieval with Ontology-Based Filtering and Implementation Support

Master's thesis in Engineering and ICT
Supervisor: Andrei Lobov
June 2026

Norwegian University of Science and Technology
Faculty of Engineering
Department of Mechanical and Industrial Engineering

NTNU

ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

PREFACE

Write the preface of your thesis here.

You may include acknowledgements and thanks as part of your preface on this page, or you may add it as a new chapter after the preface.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

List of all abbreviations in alphabetic order:

- **NTNU** Norwegian University of Science and Technology

CHAPTER

ONE

INTRODUCTION

This chapter introduces the purpose and overall direction of the thesis. It presents the motivation for the study and describes the main problem that is addressed. The research questions are then defined to guide the work. Finally, the chapter provides an overview of the structure of the thesis.

1.1 Motivation

Machine learning has seen strong growth in recent years and is now widely applied in product development and production. It is used for tasks such as optimization, control, prediction, and decision support (Wuest et al. 2016).

Advances in data availability and computing power have further increased the use of machine learning in industrial settings (Rai et al. 2021). As a result, machine learning plays an important role in improving efficiency, supporting decision-making, and enabling new capabilities in engineering and manufacturing.

At the same time, the field of machine learning is large and diverse. Many different algorithms, methods, and approaches are available, each with its own strengths and weaknesses (Wuest et al. 2016). There is no single method that performs best for all problems, and performance depends on the specific application and data (Nti et al. 2022a; I. Lee and Shin 2020).

This creates a challenge for engineers and practitioners, who must select suitable methods for their problems. The large number of available options can act as a barrier and limit the effective use of machine learning in practice (Nti et al. 2022a). Selecting an appropriate method often requires balancing multiple factors such as accuracy, interpretability, and data requirements (Plathottam et al. 2023a; Ali, S. Lee, and Chung 2017).

If the selection is not done carefully, the resulting model may perform poorly or fail to provide useful insights (Ali, S. Lee, and Chung 2017). In practice, simpler models are often preferred due to their interpretability, even if more complex models may offer higher accuracy (Paleyes, Urma, and Lawrence 2022). This highlights the importance of considering trade-offs in real-world applications.

Another challenge is the gap between research and practical implementation. Research results are distributed across many articles and domains, which makes it difficult to gain a complete overview (Nti et al. 2022a). In addition, most studies focus on developing models, while less attention is given to how they should be selected and applied in practice.

Companies also face challenges related to lack of knowledge and experience when adopting machine learning (Sonntag et al. 2023). In many cases, the selection of methods is based on trial and error, which is time-consuming and inefficient (Sonntag et al. 2023). There is also often pressure to adopt machine learning without a clear plan, which increases the risk of unsuccessful implementations (Plathottam et al. 2023a).

These challenges highlight the need for more structured approaches that can support method selection and make machine learning more accessible.

1.2 Problem Description

Despite the growing adoption of machine learning in manufacturing and engineering, practitioners often lack adequate support when selecting appropriate methods for their specific problems. Although a large number of use cases exist, there is currently no guidance or standard for developing machine learning solutions from ideation to deployment, and it remains difficult for non-experts to begin developing solutions for their specific problems (Chen et al. 2023). In practice, method selection is often based on trial and error, which is both time-consuming and dependent on prior experience that many practitioners do not have (Sonntag et al. 2023).

A central limitation is that no structured approach currently exists for making an informed selection of machine learning algorithms based on a well-defined problem description (Sonntag et al. 2023). Existing frameworks tend to use accuracy or prediction speed as the primary selection criteria, which is often too general and can lead to unsuitable methods being chosen (Sala et al. 2018). Manual evaluation of candidate algorithms is highly time-consuming, and if the selection is not done carefully, the resulting model may fail to produce useful results (Ali, S. Lee, and Chung 2017).

Existing research further focuses primarily on the development and evaluation of machine learning models, while less attention is given to what conditions lead to selecting one approach over another in specific industrial contexts (Arena et al. 2024). The lack of interpretability in many approaches is also a key barrier to adoption, as practitioners need to understand and trust the recommendations they receive (Presciuttini et al. 2024).

At the same time, the relevant knowledge is distributed across a large and fragmented body of literature. The extensive number of works on machine learning algorithms and applications creates broad knowledge on the topic, but may also generate disorientation when selecting the right approach (Sala et al. 2018). This knowledge is difficult to access and apply systematically during method selection, which limits the ability to make informed, evidence-based decisions (Nti et al. 2022a).

Addressing these challenges requires a structured approach that can translate problem descriptions into method recommendations and connect those recommendations to relevant research. This thesis investigates whether combining ontology-based knowledge representation with semantic retrieval from scientific literature can provide such support, offering interpretable recommendations grounded in prior research alongside guidance for early-stage implementation.

Burde MLguide forklares i så stor detalj? Og bør det forklares her eller under research questions?

1.3 Research Questions

The challenges outlined above point to a need for structured decision support that can connect problem descriptions with suitable machine learning methods, use the research literature to justify those recommendations, and lower the barrier to early-stage implementation. This thesis investigates whether a system combining these capabilities can provide useful and justified recommendations in practice. The following hypothesis is proposed:

> A decision-support system that combines ontology-based knowledge representation, semantic similarity between problem descriptions and scientific literature, and structured implementation support can provide relevant and justified machine learning method recommendations for users with limited machine learning expertise.

To investigate this hypothesis, the following research questions are defined:

- How can ontology-based knowledge improve machine learning method selection?
- How can semantic similarity between problem descriptions and literature support recommendations?
- How can literature-based evidence support and justify machine learning method recommendations?
- To what extent can such a system support early-stage machine learning implementation?

To investigate the hypothesis and research questions, a web-based decision-support system, called MLguide, is developed that combines structured knowledge with information from scientific literature.

1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- Chapter 1 introduces the motivation, problem description, and research questions
- Chapter 2 presents background and related work
- Chapter 3 describes the methodology
- Chapter 4 presents the system implementation
- Chapter 5 presents the results
- Chapter 6 discusses the findings and limitations
- Chapter 7 concludes the thesis and suggests future work

CHAPTER

TWO

BACKGROUND AND STATE OF THE ART

2.1 Machine Learning

- Overview of ML paradigms (supervised, unsupervised, etc.)
- Typical ML tasks (classification, regression, time series, etc.)
- Challenges in selecting appropriate methods
- Existing guidelines or frameworks for method selection

2.1.1 Machine Learning Paradigms and Tasks

Machine learning algorithms can be broadly categorized according to the type of supervision they receive during training. Mitchell (1997) defines a learning algorithm as a computer program that improves its performance at some task through experience. Three main learning paradigms are commonly distinguished in the literature: supervised learning, unsupervised learning, and reinforcement learning (Bishop 2006) (Goodfellow, Bengio, and Courville 2016, Chapter 5) (Sarker 2021). The following sections outline each paradigm and introduce some common tasks.

In supervised learning, the training data consists of input vectors paired with corresponding target values. The two most common supervised tasks are classification, where the goal is to assign an input to one of a finite number of discrete categories, and regression, where the goal is to predict a continuous numerical value (Bishop 2006) (Goodfellow, Bengio, and Courville 2016, Chapter 5).

In unsupervised learning, no target values are provided. The algorithm must instead identify structure in the data directly (Bishop 2006). Common tasks include clustering, dimensionality reduction, and anomaly detection (Sarker 2021) (Goodfellow, Bengio, and Courville 2016, Chapter 5). Clustering groups data points such that objects within the same group are more similar to each other than to those in other groups (Sarker 2021). Dimensionality reduction simplifies data by reducing the number of features, which leads to better human interpretations and lower computational costs (Sarker 2021). Anomaly detection identifies observations that deviate significantly from the expected distribution, and is used in applications such as fraud detection and fault monitoring (Goodfellow, Bengio, and Courville 2016, Chapter 5). Semi-supervised learning occupies a middle ground, operating on both labeled and unlabeled data, which is useful in contexts where labeled examples are scarce (Sarker 2021). It is worth noting that the boundary between supervised and unsupervised learning is not always well-defined, and many methods can be applied across both types of problems (Goodfellow, Bengio, and Courville 2016, Chapter 5).

Reinforcement learning differs fundamentally from the other paradigms. Rather than learning from a fixed dataset, an agent interacts with an environment and learns through trial and error, receiving rewards or penalties based on its actions (Bishop 2006) (Sarker 2021). Within reinforcement learning, methods can be broadly divided according to what they learn. In value-based methods, a value function is learned from which the policy is derived either explicitly or implicitly, whereas in policy-based methods the policy is learned directly without the need for a value function (Sewak 2019) (Sutton and Barto 2018, Chapter 11). A third class, actor-critic methods, combines both approaches by maintaining a separate policy structure and value function simultaneously (Sutton and Barto 2018, Chapter 11).

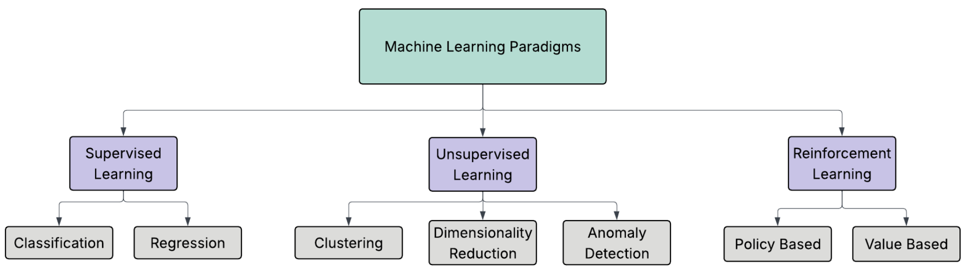

Figure 2.1.1: Overview of machine learning paradigms with common tasks.

Sett inn et delkapittel om ML metoder? F.eks: Formidle at antallet konkrete metoder er stort. Vise at metodene varierer langs noen få meningsfulle dimensjoner som faktisk har praktisk betydning: tolkbarhet, datakrav, robusthet, og ytelse Det vil gjøre begrunnelsen av Ontologi mer meningsfull, siden det er nettopp disse egenskapene som kan representeres eksplisitt i en ontologi og brukes for inferens.

2.1.2 Machine Learning Methods: Categories and Attributes

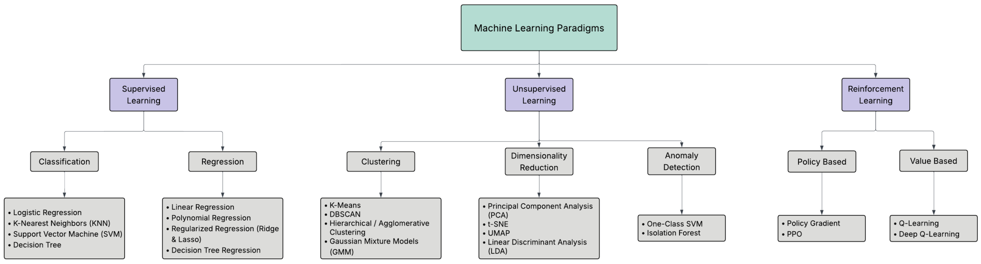

Figure 2.1.2: Overview of common machine learning methods categorized by paradigms and tasks, based on Sarker (2021), Geron (2019), and Zhang and Yu (2020).

- Explain shortly the main categories of methods (e.g., decision trees, SVMs, neural networks, etc.) and their typical use cases
- Attributes relevant for method selection:
  - Data type: tabular, time series, image, text
  - Data availability: large vs. limited data requirements
  - Interpretability: interpretable vs. black-box methods
  - Performance preferences: accuracy, training time, computational cost, prediction speed

2.1.3 Approaches to Method Selection

Selecting an appropriate machine learning method is not straightforward. As discussed in Chapter 1.1, the large number of available algorithms, each with distinct strengths and limitations, creates a real challenge for practitioners. This challenge is formally described by the Algorithm Selection Problem (Smith-Miles 2009), which asks which algorithm is likely to perform best for a given problem. The No Free Lunch theorems provide a theoretical basis for this difficulty, showing that no single algorithm performs best across all problems and that good selection requires understanding the characteristics of the problem at hand (Wolpert and Macready 1997).

Several practical tools have been developed to help with method selection. Scikit-learn provides a publicly available decision flowchart that guides users through questions about data size and task type to suggest candidate algorithms (Scikit-learn developers n.d.), and Microsoft Azure offers a similar cheat sheet for its machine learning platform (Microsoft n.d.). While these tools are accessible and widely used, they have clear limitations. The selection criteria used in both tools are mainly accuracy and prediction speed, something that can in practice lead to poor results. (Sala et al. 2018). Factors such as interpretability, data characteristics, and domain context are not considered, and neither do they offer any guidance on how the problem itself should be formulated. (Chen et al. 2023).

More structured frameworks have been proposed in the academic literature. Sala et al. (2018) present a two-layer selection framework that incorporates algorithm characteristics such as scalability and robustness to outliers, in addition to learning type and response type. Domain-specific frameworks have also been proposed, such as that of Arena et al. (2024), who develop a structured set of decision variables for algorithm selection in predictive maintenance. However, as the authors note, most existing approaches focus narrowly on accuracy and computational requirements, and none provides a tool that connects algorithmic choices to the broader context of the application domain.

A related class of approaches is automated machine learning (AutoML), which seeks to automatically select, compose, and parametrize machine learning models to achieve optimal performance on a given task or dataset (Waring, Lindvall, and Umeton 2020). AutoML systems automate parts of the machine learning pipeline, including data preparation, feature engineering, hyperparameter optimization, and model generation, with the goal of reducing the demand for expert knowledge and making machine learning more accessible to domain experts (He, Zhao, and Chu 2021). Within AutoML, the algorithm selection problem is typically treated as an optimization task, where dataset characteristics are used to predict which algorithm is likely to perform best (Barbudo, Ventura, and Romero 2023).

However, AutoML approaches have notable limitations in the context of decision support for non-experts. Most systems treat the selection and configuration process as a black box, providing little explanation of why a particular configuration was chosen (Barbudo, Ventura, and Romero 2023). This lack of transparency is a significant barrier to adoption, as users who do not understand or trust the output of a system are unlikely to apply it in practice (Waring, Lindvall, and Umeton 2020). Furthermore, AutoML primarily addresses the later stages of the machine learning pipeline, and the phases considered inherently human, such as problem formulation and interpretation of results, have received little attention (Barbudo, Ventura, and Romero 2023). This means that AutoML does not address the earlier challenge of helping practitioners understand which type of problem they have and which general approach is appropriate. This gap motivates the work in this thesis.

Nevne spesifikke AutoML verktøy som er relevante? Eller ta det i exisiting tools

2.2 Machine Learning in Product Development and Production

- Go through findings from the literature review in pre-master project
- ML is used for tasks such as Quality prediction and defect detection, Process optimization, Predictive maintenance, Time series forecasting
- Product development often has limited and uncertain data
- Production systems may have structured and continuous data

- Selecting suitable ML methods is challenging in these contexts

2.2.1 Applications of Machine Learning

Machine learning is widely applied in industrial product development and production, where it supports tasks such as process optimization, quality prediction, fault diagnosis, predictive maintenance, and anomaly detection (Wuest et al. 2016; Plathottam et al. 2023b). The application of ML in these domains continues to grow, driven by the increasing availability of production data and the improved usability of ML tools (Wuest et al. 2016). Unlike traditional rule-based approaches, ML methods can learn directly from data and adapt to changing conditions, making them well suited to the complex and dynamic nature of manufacturing environments (Wuest et al. 2016). In practice, ML is used both to improve product quality and to optimize production processes, for example through early prediction of manufacturing outcomes, root cause analysis, and condition monitoring of equipment (Weichert et al. 2019). As well as individual processes, ML also supports broader operational tasks such as production planning, scheduling, and supply chain management (Plathottam et al. 2023b; Presciuttini et al. 2024).

2.2.2 Learning Paradigms in Industrial Applications

Across reported applications in manufacturing and production, supervised learning is the dominant paradigm, accounting for the large majority of studies in most application areas (Wuest et al. 2016; Nti et al. 2022b). This reflects the nature of many industrial problems, where labeled data are available through quality inspections, sensor logs, and operator feedback, and where the goal is typically to predict a known outcome such as product quality or equipment failure (Wuest et al. 2016; Presciuttini et al. 2024). Unsupervised learning plays a smaller but important role, particularly in anomaly detection and condition monitoring, where labeled examples of faults are scarce or costly to obtain (Presciuttini et al. 2024). Reinforcement learning is less commonly reported in industrial settings, though it has shown strong results in process control and scheduling problems, where decisions must be made sequentially over time (Panzer and Bender 2022). In practice, the choice of learning paradigm is often shaped by what data is available rather than by the structure of the problem itself (Wuest et al. 2016).

2.2.3 Pre-Master Findings

In a preceding specialization project, a large-scale analysis of machine learning research in product development and production was conducted, classifying over 33,000 article abstracts across product lifecycle phase, application area, learning paradigm, and reported ML methods. The results show that the literature is strongly concentrated in the optimization phase of the product lifecycle, while early phases such as planning and concept development contain comparatively few studies. Supervised learning dominates across all phases and application areas, accounting for approximately 73–80% of the articles in each lifecycle phase, with neural networks, random forests, and support vector machines as the most frequently reported methods. Based on these findings, a roadmap was proposed that structures the literature across lifecycle phase, application area, and learning paradigm to support early-stage method selection (Bergstø 2026).

2.3 Ontologies and Knowledge Representation

- Definition of an ontology: concepts, relations, and formal structure
- Motivations for using ontologies: shared vocabulary, explicit assumptions, reusability
- Relevance for ML method selection: structuring hierarchical relationships between methods, tasks, and problem types to support interpretable recommendations
- Ontologies in the ML domain: existing ML ontologies and their properties
- Limitations: manual effort, iterative maintenance, scope constraints

2.3.1 Ontology Description

An ontology is an explicit specification of a conceptualization, where a conceptualization is an abstract model of a domain that identifies its relevant concepts and the relationships between them (Gruber 1995). Studer, Benjamins, and Fensel (1998) refine this by describing an ontology as a *formal, explicit specification of a shared conceptualization*. Each term in this definition carries a distinct meaning. *Formal* means the ontology is machine-readable and expressed in a language with precise semantics. *Explicit* means that all concepts and the constraints on their use are clearly defined, rather than left implicit in code or documentation. *Shared* means the ontology captures knowledge that is accepted by a group, rather than reflecting the perspective of a single individual (Studer, Benjamins, and Fensel 1998). In practical terms, an ontology defines a common vocabulary for a domain, including machine-interpretable definitions of its basic concepts and the relations among them (Noy and McGuinness 2001).

An ontology consists of classes, which represent concepts in the domain, properties that describe features of those concepts, and relations that link concepts to one another (Noy and McGuinness 2001). Classes are typically organized in a hierarchy where subclasses inherit the properties of their parent classes. Taken together, these elements allow a domain to be described in a way that both humans and software can interpret and reason over (Gruber 1995).

The design of an ontology involves several practical constraints. Gruber (1995) argues that an ontology should require only the minimal ontological commitment needed for its intended purpose, meaning it should make as few claims about the world as necessary. This principle reflects a trade-off identified by Studer, Benjamins, and Fensel (1998): the more reusable an ontology is, the less directly applicable it tends to be in practice, since a more general representation provides less operational guidance for a specific task. Noy and McGuinness (2001) point to a related constraint on scope. An ontology should not attempt to capture everything that could be said about a domain, but only what is needed for the application at hand. Expanding scope beyond what the application requires adds complexity without benefit, and may introduce inconsistencies that are difficult to resolve. Ontology development is also not a one-time activity. As domain knowledge evolves and new requirements emerge, the ontology must be revised, which means the initial design should anticipate extension and allow the vocabulary to be specialized without requiring changes to existing definitions (Gruber 1995; Noy and McGuinness 2001).

2.3.2 Why ML and Ontology

The machine learning domain has several properties that make ontological representation a natural fit.

ML methods are not a flat collection of independent techniques. They form hierarchical structures where more specific methods are specializations of more general ones. Humm (2026) illustrates this directly: a multilayer perceptron is a specialization of an artificial neural network, and if the latter can be used for classification and regression, the former inherits that applicability without needing it to be stated separately. This kind of reasoning over hierarchical structure is something an ontology supports naturally through inheritance, but is difficult to achieve in a flat list or a conventional database schema.

Beyond hierarchical structure, ML concepts are connected through typed semantic relations. Methods are suited to certain tasks and datatypes, belong to certain learning paradigms, and have different performance characteristics. These associations are not incidental but reflect the structure of the domain itself, and making them explicit is a prerequisite for structured method selection. Keet et al. (2015) motivate their ontology for data mining precisely on these grounds: traditional approaches treated learning algorithms as black boxes, correlating observed performance with data characteristics alone, while an ontology allows the internal characteristics of algorithms, their assumptions, optimization strategies, and decision mechanisms, to be represented explicitly and reasoned over.

2.3.3 Existing Machine Learning Ontologies

- Overview of existing ML ontologies: scope, structure, and focus
- Comparison of ontologies: coverage of methods, tasks, and data types.
- Different application domains.
- Present ML Ontology as the most comprehensive and suitable for method selection
- Show structure through diagram (Protege?) and examples of key concepts and relations

2.4 Semantic Retrieval and Text Similarity

Ontologies provide structured, explicit representations of domain knowledge, but they depend on manually defined concepts and relations. This makes them well-suited for reasoning over known entities, yet limited in their ability to handle descriptions expressed in natural language or to retrieve knowledge that was not explicitly encoded. Semantic retrieval addresses this gap by operating on the meaning of text rather than on predefined structure. This section introduces the core concepts underlying this approach: how text can be represented as numerical vectors, how similarity between texts can be measured in that representation, and how these techniques support ranking and matching against a corpus of scientific literature.

2.4.1 Text Embeddings and Semantic Similarity

- Text represented as numerical vectors (embeddings) in a high-dimensional space
- Semantically similar texts receive similar vector representations and are close in embedding space
- Cosine similarity as the primary measure for comparing embeddings

In vector semantics, text is represented as a numerical vector, a point in a high-dimensional space referred to as the embedding space (Jurafsky and Martin 2026, p. 116). The position of a vector in this space reflects the semantic content of the text it represents. A central property of this representation is that semantically similar texts are mapped to nearby points in the embedding space, while unrelated texts are placed further apart. Figure 2.4.1 illustrates this principle for a small set of example words.

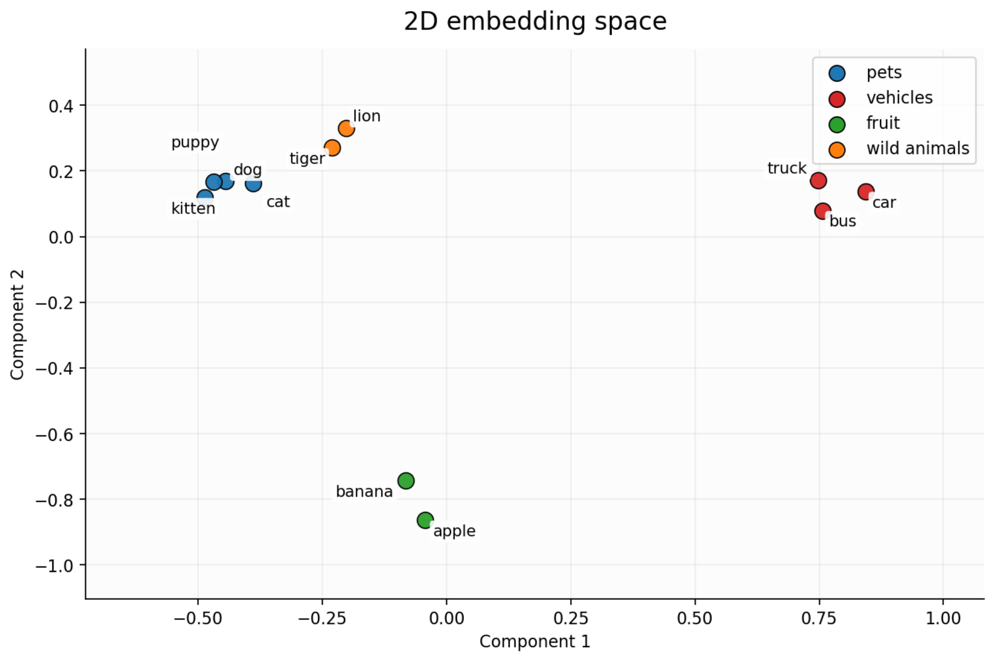

Figure 2.4.1: A two-dimensional projection of example word embeddings. Semantically similar words cluster together in the embedding space, while unrelated words are positioned further apart.

To compare two vectors $\mathbf{u}$ and $\mathbf{v}$, cosine similarity is the most common metric used in natural language processing (Jurafsky and Martin 2026, Chapter 5.4).

Rather than measuring absolute distance between two points, it measures the angle between two vectors, normalized for their length:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \tag{2.1}$$

The result ranges from 1, indicating identical direction and maximum similarity, to 0 for orthogonal vectors with no similarity (Jurafsky and Martin 2026, Chapter 5.4). Normalizing for vector length ensures that the similarity score reflects semantic content rather than vector magnitude. In practice, cosine distance, defined as $1 - \cos(\mathbf{u}, \mathbf{v})$, is often used when a distance metric is required, where lower values indicate greater similarity.

2.4.2 Transformer-Based Sentence Embedding Models

- Transformer-based language models as the foundation for contextual text representations
- SBERT (Sentence-BERT): fine-tuned to produce fixed-size sentence embeddings suitable for cosine similarity comparison
- Sentence-level embeddings suited for comparing full problem descriptions against article abstracts

Transformer-based language models represent text by processing input tokens through multiple layers of self-attention, producing contextual representations that capture relationships between words across the full input sequence (Vaswani et al. 2023). Unlike earlier embedding approaches that assigned a fixed vector to each word regardless of context, transformer models produce representations that reflect the meaning of a word given its surrounding text.

However, standard transformer models such as BERT are not directly suited for computing sentence-level similarity. As noted by Reimers and Gurevych (2019), comparing two sentences with BERT requires feeding both into the network simultaneously, which becomes computationally infeasible at scale. Finding the most similar pair in a collection of 10,000 sentences would require approximately 50 million inference computations.

Sentence-BERT (SBERT) addresses this by fine-tuning BERT using a siamese network structure, producing fixed-size sentence embeddings that can be compared efficiently using cosine similarity (Reimers and Gurevych 2019). The key property is that semantically similar sentences are placed close together in the resulting embedding space, making SBERT suitable for tasks where embeddings for a large corpus can be precomputed and a new query compared against all of them in milliseconds.

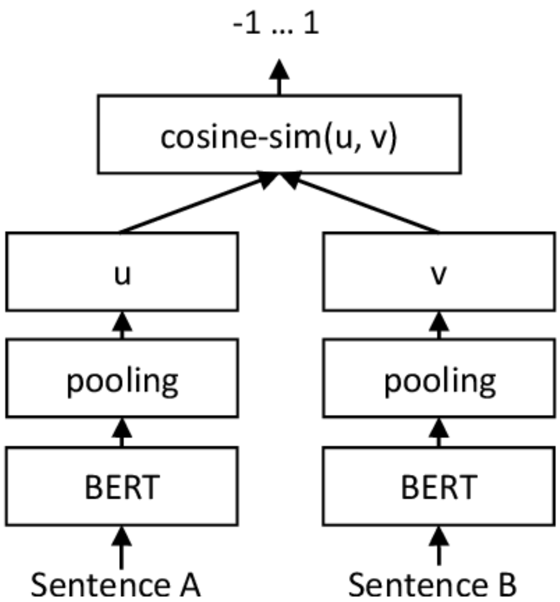

Figure 2.4.2: SBERT architecture at inference time. Each sentence is encoded independently through a shared BERT network, producing vectors **u** and **v** that are compared using cosine similarity. Adapted from Reimers and Gurevych (2019) under CC BY-SA 4.0.

The sentence-transformers library provides pretrained models based on this approach for encoding text into fixed-size vectors (*Sentence-Transformers Documentation* 2026).

2.4.3 Semantic Retrieval

- Retrieval based on semantic meaning rather than exact keyword matching
- A query is embedded and compared against a corpus of pre-embedded documents
- More flexible than keyword search: captures related concepts even when exact terms differ

Information retrieval (IR) is the task of identifying and ranking documents from a collection according to their relevance to a user query (Jurafsky and Martin 2026, Chapter 11.1). Traditional approaches represent both documents and queries as sparse vectors of word counts, weighted by schemes such as tf-idf or BM25, and compute similarity based on shared terms (Jurafsky and Martin 2026, Chapter 11.1). While effective for many tasks, this approach treats documents as bags of words, meaning that word order and meaning are ignored, and only exact term matches contribute to the relevance score.

Dense retrieval addresses this limitation by representing documents and queries as embeddings computed from language models, rather than as word count vectors (Jurafsky and Martin 2026, Chapter 11.1). Because semantically similar texts are mapped to nearby points in the embedding space, a query can retrieve relevant documents even when they do not share exact terms with the query. This makes dense retrieval more flexible than keyword-based approaches for tasks where the user's information need is expressed in natural language rather than precise search terms.

Figure 2.4.3 illustrates the distinction with a constructed example. The same query returns only Document A under keyword-based retrieval, as it is the only document sharing exact terms with the query. Dense retrieval ranks all three documents by semantic similarity, including Documents B and C despite the absence of exact term overlap in these documents.

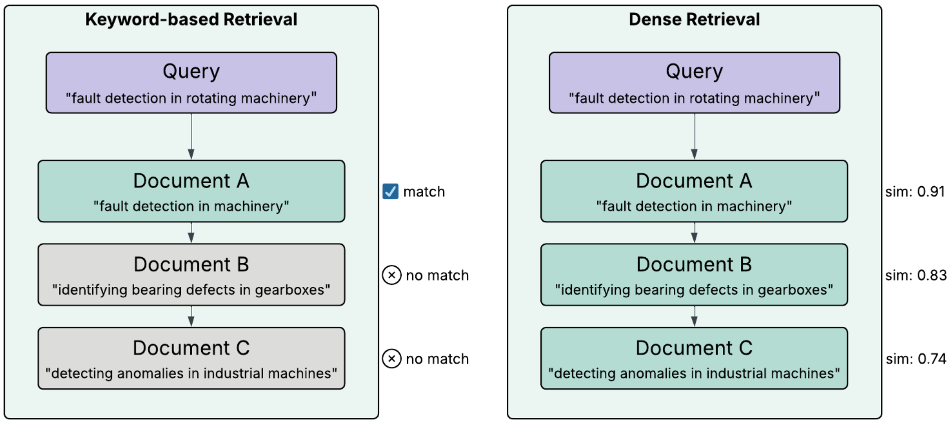

Figure 2.4.3: Keyword-based versus dense retrieval for the same query.

2.4.4 Cross-Encoder Reranking

- Bi-encoders embed query and document independently, enabling efficient large-scale comparison
- Cross-encoders process query and document jointly, producing more precise relevance scores
- Two-stage pipeline: bi-encoder retrieval for speed, cross-encoder reranking for precision
- Trade-off: cross-encoders are more accurate but too slow for large-scale initial retrieval

The sentence embedding approach described in Section **??** is an example of a bi-encoder, where query and document are encoded independently into fixed-size vectors and similarity is measured using cosine similarity (Reimers and Gurevych 2019). Because the representations are computed independently, document embeddings can be precomputed and stored, making bi-encoders efficient for large-scale retrieval (Thakur et al. 2021).

A cross-encoder takes a different approach, concatenating query and document into a single input and processing both jointly, which allows the model to apply attention across both texts simultaneously (Reimers and Gurevych 2019). This produces a direct relevance score rather than a similarity between independent vectors. Cross-encoders are generally more accurate than bi-encoders, but since no independent representations are computed, every query-document pair must be processed from scratch at query time, making cross-encoders impractical for retrieval over large document collections (Thakur et al. 2021).

Figure 2.4.4 illustrates the architectural difference between the two approaches.

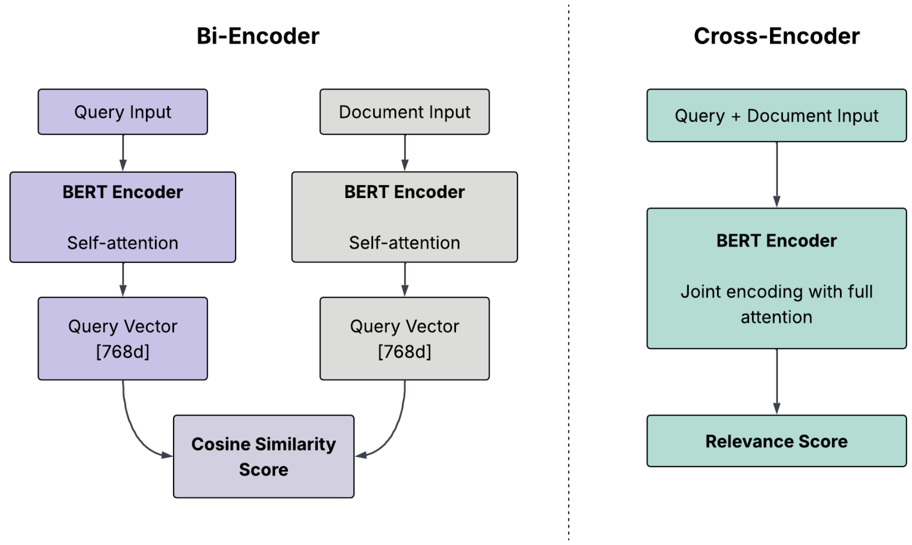

Figure 2.4.4: Bi-encoder and cross-encoder architectures. The bi-encoder encodes query and document independently, producing vectors that are compared using cosine similarity. The cross-encoder concatenates both inputs and processes them jointly, enabling full attention between query and document inputs and producing a direct relevance score.

A common approach to combining the strengths of both architectures is a two-stage retrieval pipeline (Nogueira and Cho 2019). In the first stage, a bi-encoder retrieves a set of candidate documents from the full corpus based on embedding similarity. In the second stage, a cross-encoder reranks these candidates by scoring each document against the query directly, producing a more precise relevance ranking (Nogueira and Cho 2019). This approach preserves the scalability of bi-encoder retrieval while benefiting from the higher accuracy of cross-encoder scoring.

2.4.5 Scientific Literature as a Knowledge Source

- Scientific articles as a source of evidence for machine learning method recommendations
- Large volumes of unstructured literature make manual navigation impractical
- Semantic retrieval enables systematic matching between problem descriptions and relevant articles

Scientific articles document how machine learning methods have been applied across a wide range of problems and contexts. In the domain of machine learning for engineering and manufacturing, a large number of such articles exist, covering diverse algorithms and applications (Sala et al. 2018). This breadth of literature is itself a challenge. Relevant knowledge is spread across journals, conference proceedings, and research communities (Nti et al. 2022a), and the volume of available work makes it difficult for any individual to draw connections across topics and domains (Olivetti et al. 2020). The result is that the literature creates extensive knowledge on the topic, while at the same time making the selection of the right approach difficult (Sala et al. 2018).

Semantic retrieval offers a way to address this challenge by enabling systematic matching between a problem description and relevant articles. Article abstracts are a practical textual basis for this purpose. Text mining of scientific literature has mostly been carried out on abstracts due to their availability, and the fact that abstracts consists of shorter sentences presenting only the most important findings of a study (Westergaard et al. 2018). While full-text articles contain richer information, they also include complex tables, references, and speculative discussion sections, making them less consistent as a basis for comparison across a large corpus (Westergaard et al. 2018). Abstracts thus represent a structured and consistently available representation of each article's core content.

2.5 Related Work and Existing Tools

- AutoML: Auto-sklearn, TPOT, H2O AutoML, Google AutoML
- Meta-learning: algorithm selection based on dataset characteristics and prior experience
- OMA-ML ?
- Traingenerator: template-based notebook generation
- AI on Demand

CHAPTER

THREE

METHODOLOGY

CHAPTER FOUR

IMPLEMENTATION

CHAPTER

FIVE

RESULTS

CHAPTER

SIX

DISCUSSION

CHAPTER

SEVEN

CONCLUSIONS

REFERENCES

Wuest, Thorsten et al. (2016). "Machine learning in manufacturing: advantages, challenges, and applications". In: *Production & Manufacturing Research* 4.1. doi: 10.1080/21693277.2016.1192517, pp. 23–45. ISSN: null. DOI: 10.1080/21693277.2016.1192517. URL: https://doi.org/10.1080/21693277.2016.1192517.

Rai, Rahul et al. (2021). "Machine learning in manufacturing and industry 4.0 applications". In: *International Journal of Production Research* 59.16. doi: 10.1080/00207543.2021.1956675, pp. 4773–4778. ISSN: 0020-7543. DOI: 10.1080/00207543.2021.1956675. URL: https://doi.org/10.1080/00207543.2021.1956675.

Nti, Isaac Kofi et al. (2022a). "Applications of artificial intelligence in engineering and manufacturing: a systematic review". In: *Journal of Intelligent Manufacturing* 33.6, pp. 1581–1601. ISSN: 1572-8145. DOI: 10.1007/s10845-021-01771-6. URL: https://doi.org/10.1007/s10845-021-01771-6.

Lee, In and Yong Jae Shin (2020). "Machine learning for enterprises: Applications, algorithm selection, and challenges". In: *Business Horizons* 63.2, pp. 157–170. ISSN: 0007-6813. DOI: https://doi.org/10.1016/j.bushor.2019.10.005. URL: https://www.sciencedirect.com/science/article/pii/S0007681319301521.

Plathottam, Siby Jose et al. (2023a). "A review of artificial intelligence applications in manufacturing operations". In: *Journal of Advanced Manufacturing and Processing* 5.3, e10159. DOI: https://doi.org/10.1002/amp2.10159. URL: https://doi.org/10.1002/amp2.10159.

Ali, Rahman, Sungyoung Lee, and Tae Choong Chung (2017). "Accurate multi-criteria decision making methodology for recommending machine learning algorithm". In: *Expert Systems with Applications* 71, pp. 257–278. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2016.11.034. URL: https://www.sciencedirect.com/science/article/pii/S0957417416306698.

Paleyes, Andrei, Raoul-Gabriel Urma, and Neil D. Lawrence (2022). "Challenges in Deploying Machine Learning: A Survey of Case Studies". In: *ACM Comput. Surv.* 55.6, Article 114. ISSN: 0360-0300. DOI: 10.1145/3533378. URL: https://doi.org/10.1145/3533378.

Sonntag, Sebastian et al. (2023). "A PATTERN LANGUAGE APPROACH TO IDENTIFY APPROPRIATE MACHINE LEARNING ALGORITHMS IN THE CONTEXT OF PRODUCT DEVELOPMENT". In: *Proceedings of the Design*

Society 3, pp. 365–374. DOI: 10.1017/pds.2023.37. URL: https://www.cambridge.org/core/product/44BBE29AA01344BC5F50174953C5BA8B.

Chen, Tingting et al. (2023). *Machine Learning in Manufacturing towards Industry 4.0: From 'For Now' to 'Four-Know'*. Electronic Article. DOI: 10.3390/app13031903.

Sala, Roberto et al. (2018). *How to select the right machine learning algorithm: a feature-based, scope-oriented selection framework*.

Arena, Simone et al. (2024). "A conceptual framework for machine learning algorithm selection for predictive maintenance". In: *Engineering Applications of Artificial Intelligence* 133, p. 108340. ISSN: 0952-1976. DOI: https://doi.org/10.1016/j.engappai.2024.108340. URL: https://www.sciencedirect.com/science/article/pii/S0952197624004986.

Presciuttini, Anna et al. (2024). "Machine learning applications on IoT data in manufacturing operations and their interpretability implications: A systematic literature review". In: *Journal of Manufacturing Systems* 74, pp. 477–486. ISSN: 0278-6125. DOI: https://doi.org/10.1016/j.jmsy.2024.04.012. URL: https://www.sciencedirect.com/science/article/pii/S0278612524000773.

Mitchell, Tom M. (1997). *Machine Learning*. New York: McGraw-Hill.

Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer. ISBN: 978-0387310732.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. http://www.deeplearningbook.org. MIT Press.

Sarker, Iqbal H. (2021). "Machine Learning: Algorithms, Real-World Applications and Research Directions". In: *SN Computer Science* 2.3, p. 160. ISSN: 2661-8907. DOI: 10.1007/s42979-021-00592-x. URL: https://doi.org/10.1007/s42979-021-00592-x.

Sewak, Mohit (2019). "Policy-Based Reinforcement Learning Approaches". In: *Deep Reinforcement Learning: Frontiers of Artificial Intelligence*. Ed. by Mohit Sewak. Singapore: Springer Singapore, pp. 127–140. ISBN: 978-981-13-8285-7. DOI: 10.1007/978-981-13-8285-7_10. URL: https://doi.org/10.1007/978-981-13-8285-7_10.

Sutton, Richard S. and Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA: MIT Press.

Geron, Aurelien (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Inc. ISBN: 1492032646.

Zhang, Hongming and Tianyang Yu (2020). "Taxonomy of Reinforcement Learning Algorithms". In: *Deep Reinforcement Learning: Fundamentals, Research and Applications*. Ed. by Hao Dong, Zihan Ding, and Shanghang Zhang. Singapore: Springer Singapore, pp. 125–133. ISBN: 978-981-15-4095-0. DOI: 10.1007/978-981-15-4095-0_3. URL: https://doi.org/10.1007/978-981-15-4095-0_3.

Smith-Miles, Kate A. (2009). "Cross-Disciplinary Perspectives on Meta-Learning for Algorithm Selection". In: *ACM Computing Surveys* 41.1. DOI: 10.1145/1456650.1456656.

Wolpert, David H. and William G. Macready (1997). "No Free Lunch Theorems for Optimization". In: *IEEE Transactions on Evolutionary Computation* 1.1, pp. 67–82. DOI: 10.1109/4235.585893.

Scikit-learn developers (n.d.). *Choosing the right estimator*. Accessed: May 2026. URL: https://scikit-learn.org/stable/machine_learning_map.html.

Microsoft (n.d.). *Machine Learning Algorithm Cheat Sheet for Azure Machine Learning*. Accessed: May 2026. URL: https://learn.microsoft.com/en-us/azure/machine-learning/algorithm-cheat-sheet.

Waring, Jonathan, Charlotta Lindvall, and Renato Umeton (2020). “Automated machine learning: Review of the state-of-the-art and opportunities for healthcare”. In: *Artificial Intelligence in Medicine* 104, p. 101822. ISSN: 0933-3657. DOI: https://doi.org/10.1016/j.artmed.2020.101822. URL: https://www.sciencedirect.com/science/article/pii/S0933365719310437.

He, Xin, Kaiyong Zhao, and Xiaowen Chu (2021). “AutoML: A survey of the state-of-the-art”. In: *Knowledge-Based Systems* 212, p. 106622. ISSN: 0950-7051. DOI: https://doi.org/10.1016/j.knosys.2020.106622. URL: https://www.sciencedirect.com/science/article/pii/S0950705120307516.

Barbudo, Rafael, Sebastián Ventura, and José Raúl Romero (2023). “Eight years of AutoML: categorisation, review and trends”. In: *Knowledge and Information Systems* 65.12, pp. 5097–5149. ISSN: 0219-3116. DOI: 10.1007/s10115-023-01935-1. URL: https://doi.org/10.1007/s10115-023-01935-1.

Plathottam, Siby Jose et al. (2023b). “A review of artificial intelligence applications in manufacturing operations”. In: *Journal of Advanced Manufacturing and Processing* 5.3, e10159. DOI: https://doi.org/10.1002/amp2.10159. URL: https://doi.org/10.1002/amp2.10159.

Weichert, Dorina et al. (2019). “A review of machine learning for the optimization of production processes”. In: *The International Journal of Advanced Manufacturing Technology* 104.5, pp. 1889–1902. ISSN: 1433-3015. DOI: 10.1007/s00170-019-03988-5. URL: https://doi.org/10.1007/s00170-019-03988-5.

Nti, Isaac Kofi et al. (2022b). “Applications of artificial intelligence in engineering and manufacturing: a systematic review”. In: *Journal of Intelligent Manufacturing* 33.6, pp. 1581–1601. ISSN: 1572-8145. DOI: 10.1007/s10845-021-01771-6. URL: https://doi.org/10.1007/s10845-021-01771-6.

Panzer, Marcel and Benedict Bender (2022). “Deep reinforcement learning in production systems: a systematic literature review”. In: *International Journal of Production Research* 60.13. doi: 10.1080/00207543.2021.1973138, pp. 4316–4341. ISSN: 0020-7543. DOI: 10.1080/00207543.2021.1973138. URL: https://doi.org/10.1080/00207543.2021.1973138.

Bergstø, Martin (Jan. 2026). “Supporting Method Selection for Machine Learning in Product Development and Production: A Multi-Dimensional Classification of Machine Learning Research”. In: Specialization Project.

Gruber, Thomas R. (1995). “Toward principles for the design of ontologies used for knowledge sharing?” In: *International Journal of Human-Computer Studies* 43.5, pp. 907–928. ISSN: 1071-5819. DOI: https://doi.org/10.1006/ijhc.1995.1081. URL: https://www.sciencedirect.com/science/article/pii/S1071581985710816.

Studer, Rudi, V. Richard Benjamins, and Dieter Fensel (1998). “Knowledge engineering: Principles and methods”. In: *Data & Knowledge Engineering* 25.1, pp. 161–197. ISSN: 0169-023X. DOI: https://doi.org/10.1016/S0169-

023X(97)00056-6. URL: https://www.sciencedirect.com/science/article/pii/S0169023X97000566.

Noy, N. and Deborah McGuinness (2001). “Ontology Development 101: A Guide to Creating Your First Ontology”. In: *Knowledge Systems Laboratory* 32.

Humm, Bernhard (2026). “An Industry-Ready Machine Learning Ontology”. In: *Applied Sciences* 16, p. 843. DOI: 10.3390/app16020843.

Keet, C. Maria et al. (2015). “The Data Mining OPtimization Ontology”. In: *Journal of Web Semantics* 32, pp. 43–53. ISSN: 1570-8268. DOI: https://doi.org/10.1016/j.websem.2015.01.001. URL: https://www.sciencedirect.com/science/article/pii/S1570826815000025.

Jurafsky, Daniel and James H. Martin (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd. Online manuscript released January 6, 2026. URL: https://web.stanford.edu/~jurafsky/slp3/.

Vaswani, Ashish et al. (2023). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

Reimers, Nils and Iryna Gurevych (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, pp. 3973–3983. DOI: 10.18653/v1/D19-1410.

Sentence-Transformers Documentation (2026). URL: https://www.sbert.net/ (visited on 05/11/2026).

Thakur, Nandan et al. (2021). *Augmented SBERT: Data Augmentation Method for Improving Bi-Encoders for Pairwise Sentence Scoring Tasks*, pp. 296–310. DOI: 10.18653/v1/2021.naacl-main.28.

Nogueira, Rodrigo and Kyunghyun Cho (2019). *Passage Re-ranking with BERT*.

Olivetti, Elsa et al. (2020). “Data-driven materials research enabled by natural language processing and information extraction”. In: *Applied Physics Reviews* 7, p. 041317. DOI: 10.1063/5.0021106.

Westergaard, David et al. (2018). “A comprehensive and quantitative comparison of text-mining in 15 million full-text articles versus their corresponding abstracts”. In: *PLOS Computational Biology* 14.2, e1005962. DOI: 10.1371/journal.pcbi.1005962. URL: https://doi.org/10.1371/journal.pcbi.1005962.

APPENDICES